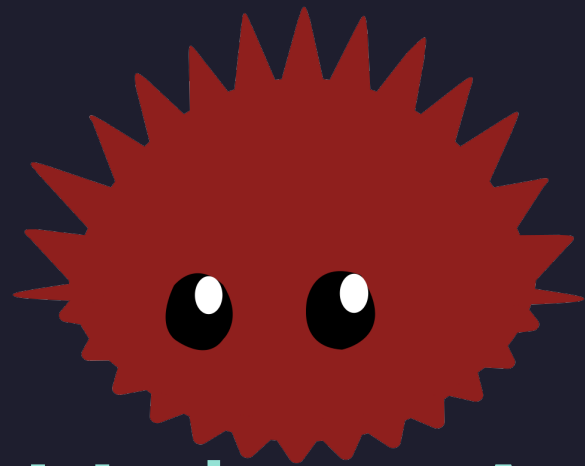
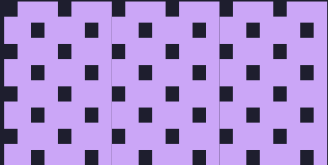


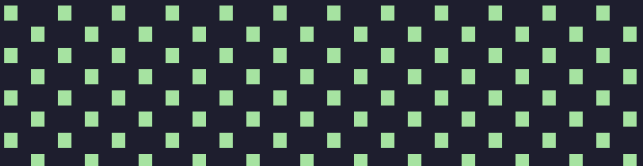


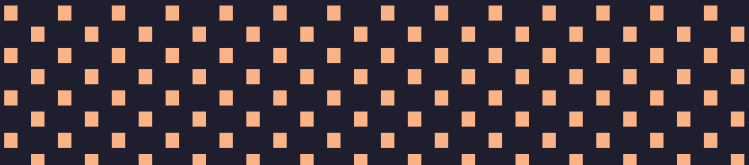
(ab)Using Advent of Code as an FFI Playground

 [Learning in Public]

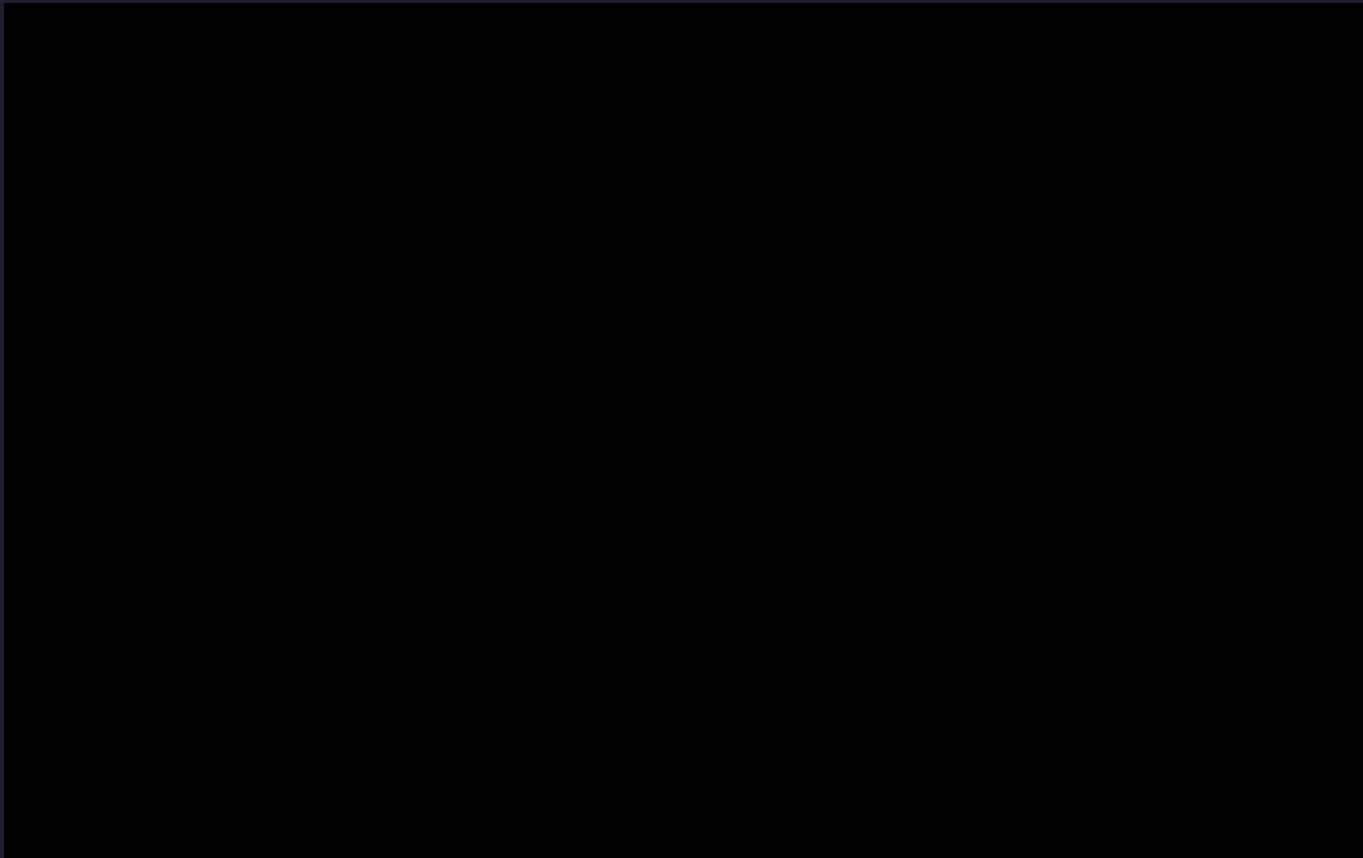
 @ Rust Los Angeles

 Santa Monica, CA

 2026-01-28

 Alyssa Evans

Who am I?



Staff Software Engineer at Ditto (<https://ditto.com>)

~~Learning in Public : AOC FFI Playground~~



Who am I?



Staff Software Engineer at Ditto (<https://ditto.com>)

and Motorcycle Road Racer



Why Rust?



Advent of Code → Rust FFI



I like winning!



Advent of Code → Rust FFI

Advent of Code → Rust FFI

Step-by-step exploration

1. Pure Rust solution
2. C FFI with qsort
3. GLib integration
4. Benchmarking with Criterion
5. UniFFI (python/kotlin/swift)

Bonus

- Zero-cost abstractions

Day 1: Historian Hysteria

Problem

Given two lists of numbers:

- Parse input into two separate lists
- Sort both lists
- Calculate sum of absolute differences

Input:

3 4

4 3

2 5

Output: Sum of $|3-3| + |4-4| + |2-5| = 3$

Parse Input

```
fn unzip(input: &str) -> (Vec<i32>, Vec<i32>) {  
    input.lines()  
        .map(|line| {  
            let nums = line.split_whitespace()  
                .map(|x| x.parse::<i32>().unwrap())  
                .collect::<Vec<i32>>();  
            (nums[0], nums[1])  
        })  
        .unzip()  
}
```

Solution

```
fn process(input: &str) -> Result<String, String> {  
    let (mut left, mut right) = unzip(input);  
  
    left.sort();  
    right.sort();  
  
    let output = left.iter().zip(right.iter())  
        .map(|(l, r)| (l - r).abs())  
        .sum::<i32>();  
  
    Ok(output.to_string())  
}
```

Enter C FFI

Why use C's qsort?

- Learn FFI basics
- Compare performance with Rust
- Demonstrate unsafe code patterns

```
use std::ffi::c_void;
use std::os::raw::c_int;

unsafe extern "C" {
    fn qsort(
        base: *mut c_void,
        num: usize,
        size: usize,
        compar: unsafe extern "C" fn(*const c_void,
                                     *const c_void) -> c_int,
    );
}
```

Compare Fn

C expects a comparison function:

```
unsafe extern "C" fn compare_i32(
    a: *const c_void,
    b: *const c_void
) -> c_int {
    unsafe {
        let a = *(a as *const i32);
        let b = *(b as *const i32);
        a.cmp(&b) as c_int
    }
}
```

Enter C FFI

Wrapper Fn

Why wrap it?

- Encapsulates unsafe FFI call
- Provides ergonomic Rust API (`&mut Vec<i32>`)
- Handles pointer/size boilerplate
- Contains `unsafe` to single location

```
fn c_qsort(vec: &mut Vec<i32>) {  
    unsafe {  
        qsort(  
            vec.as_mut_ptr() as *mut c_void,  
            vec.len(),  
            std::mem::size_of::<i32>(),  
            compare_i32  
        );  
    }  
}
```

Diff

```
- left.sort();  
- right.sort();  
+ c_qsort(&mut left);  
+ c_qsort(&mut right);
```

Key points:

- `extern "C"` ABI compatibility
- Raw pointer manipulation
- Unsafe block required

Benchmarks

Sample Input (criterion)

```
process c qsort      time: [238.81 ns 239.68 ns 240.62 ns]
process rust sort    time: [178.21 ns 178.88 ns 179.67 ns]
```

re-run

```
Running benches/aoc_bench.rs (target/release/deps/aoc_bench-e5fedc316ff25a5f)
Gnuplot not found, using plotters backend
process c qsort      time: [230.71 ns 231.45 ns 232.23 ns]
                        change: [-3.6254% -3.1719% -2.6822%] (p = 0.00 < 0.05)
                        Performance has improved.
Found 4 outliers among 100 measurements (4.00%)
  1 (1.00%) low mild
  1 (1.00%) high mild
  2 (2.00%) high severe
process rust sort    time: [176.38 ns 177.38 ns 178.44 ns]
                        change: [-1.5001% -1.0515% -0.5724%] (p = 0.00 < 0.05)
                        Change within noise threshold.
Found 4 outliers among 100 measurements (4.00%)
  3 (3.00%) high mild
  1 (1.00%) high severe
```

Rust's sort is ~25% faster on this sample input

- C's `qsort` uses function pointer indirection (no inlining)
- Rust's `.sort()` monomorphizes the comparator
- FFI call overhead adds up per comparison

Part 2: Similarity Score

Problem

Given the same two lists:

- For each number in the left list, count how many times it appears in the right list
- Multiply the number by its count
- Sum all the products

Input:

3 4 3 appears 3x in right $\rightarrow 3 * 3 = 9$

4 3 4 appears 1x in right $\rightarrow 4 * 1 = 4$

2 5 2 appears 0x in right $\rightarrow 2 * 0 = 0$

Output: $9 + 4 + 0 = 13$

HashMap Solution — $O(n)$ instead of $O(n*m)$

```
use std::collections::HashMap;

pub fn process_part_2_hashmap(input: &str) -> Result<i32, String> {
    let (left, right): (Vec<i32>, Vec<i32>) = unzip(input);

    let counts: HashMap<i32, usize> = right.iter()
        .fold(HashMap::new(), |mut acc, &n| {
            *acc.entry(n).or_insert(0) += 1;
            acc
        });

    Ok(left.iter()
        .map(|&n| n * *counts.get(&n).unwrap_or(&0) as i32)
        .sum())
}
```

But Big-O doesn't tell the whole story — hash overhead can dominate on small inputs

Benchmarks: At Scale

Real Input — 1000 lines (criterion)

Part 1: Sorting

process rust sort	time: [39.414 μs 39.554 μs 39.701 μs]
process c qsort	time: [68.126 μs 68.518 μs 68.917 μs]

Rust still ~73% faster at scale.

Part 2: Frequency Counting

part 2 hashmap	time: [59.067 μs 59.269 μs 59.478 μs]
part 2 naive	time: [77.219 μs 77.426 μs 77.664 μs]

At scale, $O(n)$ HashMap wins — 30% faster than naive $O(n*m)$

Swapping the hash function: ahash

part 2 ahash	time: [44.199 μs 44.347 μs 44.500 μs]
part 2 hashmap (SipHash)	time: [56.771 μs 56.998 μs 57.234 μs]

ahash is ~29% faster — same API, zero **unsafe**, drop-in replacement

- The real bottleneck was the hash function, not the language
- Always benchmark YOUR data

UniFFI: From Rust to Every Language

🦋 Mozilla's UniFFI generates bindings for Python, Kotlin, Swift, Ruby

All our FFI so far: Rust calling C/C++. UniFFI flips it: other languages calling Rust.

🦋 Setup

```
# Cargo.toml
[lib]
crate-type = ["cdylib", "rlib"] # Shared library for FFI
name = "aoc_ffi_day01"
```

```
[dependencies]
uniffi = "0.28"
thiserror = "1.0" # Required for UniFFI error types
```

```
[build-dependencies]
uniffi = { version = "0.28", features = ["build"] }
```

Caveat: UniFFI 0.28 requires edition 2021 (not 2024)

🦋 UDL + Proc Macros

```
// src/aoc_ffi_day01.udl — minimal, functions defined via macros
namespace aoc_ffi_day01 {;
```

```
// build.rs — generate scaffolding from UDL
uniffi::generate_scaffolding("src/aoc_ffi_day01.udl");
```

```
// lib.rs — include generated code
uniffi::include_scaffolding!("aoc_ffi_day01");
```

UniFFI: From Rust to Every Language

Exporting Functions

```
// UniFFI needs owned types (String, not &str) and its own error type
#[derive(Debug, thiserror::Error, uniffi::Error)]
#[uniffi(flat_error)]
pub enum AocError {
    #[error("Parse error: {0}")]
    ParseError(String),
}

#[uniffi::export]
pub fn uniffi_process_rust_sort(input: String) -> Result<i32, AocError> {
    process_rust_sort(&input).map_err(Into::into)
}
```

Key constraints:

- No `&str` — UniFFI needs owned `String` across FFI boundary
- Custom error type with `thiserror` + `uniffi::Error`
- Thin wrappers delegate to existing implementations

Python Bindings via UniFFI

 Generate and call Rust from Python

 Generate bindings

```
# Build the shared library
cargo build --release --lib

# Generate Python module from UDL + compiled library
uniffi-bindgen generate src/aoc_ffi_day01.udl \
  --library target/release/libaoc_ffi_day01.so \
  --language python \
  --out-dir bindings/python
```

UniFFI produces a `.py` module that loads the `.so` via `ctypes`.

 Calling Rust from Python

```
import aoc_ffi_day01

sample = "3 4\n4 3\n2 5\n1 3\n3 9\n3 3"

# Part 1 — all three sorting backends
rust_sort = aoc_ffi_day01.uniffi_process_rust_sort(sample) # 11
c_qsort = aoc_ffi_day01.uniffi_process_c_qsort(sample) # 11
cpp_sort = aoc_ffi_day01.uniffi_process_cpp_sort(sample) # 11

# Part 2 — hash table backends
naive = aoc_ffi_day01.uniffi_process_part_2(sample) # 31
hashmap = aoc_ffi_day01.uniffi_process_part_2_hashmap(sample) # 31
uthash = aoc_ffi_day01.uniffi_process_part_2_uthash(sample) # 31
```

Python Bindings via UniFFI

 Error handling crosses the FFI boundary

```
try:
    result = aoc_ffi_day01.uniffi_process_rust_sort(bad_input)
except aoc_ffi_day01.AocError as e:
    print(f"Rust error caught in Python: {e}")
```

What UniFFI gives us:

- Python module auto-generated from Rust types
- Rust `Result<T, E>` → Python exceptions
- No hand-written C bindings or `ctypes` declarations
- Also generates Kotlin (JNA), Swift, and Ruby — same shared library, one command each

Lessons Learned

1. Cross-platform FFI requires platform awareness

- Library extensions differ: `.so` / `.dylib` / `.dll`
- Build artifacts isolation needed (Docker volumes for devcontainers)
- Dynamic detection at build time via `just` — not runtime

2. Tooling integration is complex

- Python: Nix's immutable `/nix/store` blocks pip — requires `venv --copies`
- Kotlin: JNA doesn't search current directory — needs `-J-Djna.library.path=.`
- UniFFI: Generated APIs don't match assumptions — always inspect bindings

3. Automation saves sanity

- `just` for task orchestration across languages
- `Nix` for reproducible environments
- Document everything in `CLAUDE.md`

4. Performance surprises

- Pure Rust usually wins (monomorphization, inlining)
- Hash function choice matters more than language (ahash > SipHash)
- Always benchmark YOUR data

5. FFI makes you resilient

- Pragmatic bridge for legacy integration
- Choose tools based on context, not dogma

Key Takeaways

1. Isolate per platform — Docker volumes, Nix, and gitignored artifacts keep macOS/Linux clean
2. Automate everything — Just + Nix + Direnv eliminated "works on my machine"
3. Document thoroughly — [CLAUDE.md](#) saved us (and Claude) multiple times
4. Inspect generated code — UniFFI transforms names and error types; verify, don't assume
5. Rust's ecosystem delivers — Fast core, battle-tested FFI, and zero-boilerplate bindings via UniFFI

Final Thoughts

▣ Cross-platform FFI development requires:

- Patience with tooling quirks (looking at you, JNA)
- Deep understanding of each platform's conventions
- Robust automation to catch platform-specific issues
- Clear documentation for context switching
- Willingness to debug cryptic errors at 3 AM

▣ The juice is worth the squeeze

When you need native performance across multiple languages and platforms, FFI is still the best tool for the job.

Just be ready for the journey.

What's Next?

Open Source Contributions

- LearnXinY: Filling documentation gaps for Dart/Rust FFI
 - Draft PR (<https://github.com/adambard/learnxinyminutes-docs/pulls/alycda>)

Ongoing Work

- Swift in DevContainers: Solving the glibc conflict (PR #5 (<https://github.com/alycda/aoc-ffi/pull/5>))
 - Docker-based testing to work around Nix glibc incompatibility
- evaluating Ditto's safer_ffi (https://github.com/getditto/safer_ffi)
 - Macro-based FFI without UDL, auto-generates C headers

Upcoming Talks

- Ditto's CRDTs: Distributed AoC solving
- Stay tuned for t chapter of my

Questions?



Thank You!

Alyssa Evans

Staff Software Engineer @ Ditto (<https://ditto.com>)

Socials

- github.com/alycda
- [linkedin.com/in/alyda](https://www.linkedin.com/in/alyda)



Resources:

- AdventOfCode (<https://adventofcode.com>)
- Learn X in Y Minutes PRs:
<https://github.com/adambard/learnxinyminutes-docs/pulls/alycda>
- This presentation:
<https://github.com/alycda/aoc-ffi/blob/main/slides.md>

Errata

"No piece of writing is ever finished.
It's just due."

--- Bill Condon

Tools Used

- Presenterm (<https://mfontanini.github.io/presenterm/>) — terminal-based slideshow (contributed PR #826 (<https://github.com/mfontanini/presenterm/pull/826>))
- Ferris (<https://rustacean.net/>) — unofficial Rust mascot
- Nix (<https://NixOS.org>) — reproducible dev environments
- Jujutsu (<https://www.jj-vcs.dev/>) — next-gen VCS (Git-compatible)
- Bacon (<https://dystroy.org/bacon/>) — background Rust code checker
- Workflowy (<https://workflowy.com>) — outlining and organizing thoughts
- Claude Code (<https://claude.ai/claude-code>) — AI pair programming

This Talk

Code & Slides: github.com/alycda/aoc-ffi (
<https://github.com/alycda/aoc-ffi>)